РЕГЛАМЕНТ КОМАНДЫ

Работа с Git и защита перед жюри

Параллельная разработка, Git, интеграционные проверки и самостоятельная оценка жюри.

20 октября 2026
Согласованная спецификация

Команда 3-5 человек
Окно 48-72 часа

Приложение не реализовано. Статические проверки документации и контрактов отделены от будущих испытаний runtime.

Практический регламент для команды 3-5 человек на 48-72 часа. Документ применяется к будущему репозиторию приложения. Этот комплект содержит спецификацию; ни репозиторий, ни PR, ни доступ жюри автоматически не создавались.

Часть I. Разработка

1. Один репозиторий и владельцы

Единый monorepo: frontend/, backend/, ml/, contracts/, deploy/, demo/, tests/, docs/. Корневой README объясняет запуск, а не историю проектирования. Все API берутся из contracts/, generated-копии не редактируются. Большие веса и исходный исследовательский dataset не коммитятся: отдельный release bundle с hashes/licences, публичным бесплатным способом получения и offline-копией для показа.

Роли: BACKEND владеет бизнес-переходами, таблицами и миграциями; FRONTEND - UX; ML - transforms/models/KB retrieval; INTEGRATION - контракты, Docker и общая поставка; QA_DEMO - независимая проверка и защита. Для 3 человек INTEGRATION/QA распределяются между backend/frontend, ML остаётся отдельным владельцем. Для 4 QA_DEMO совмещает frontend с peer-проверкой backend; для 5 роли разделяются. У автора нет права единолично принимать собственный критичный PR/модельный релиз.

На старте записать реальные имена в CODEOWNERS и таблицу ролей. До этого .example не считается настроенной защитой. API, shared DTO, migration numbering, dependency locks и compose изменяются с review владельца; feature-ветка не «захватывает» все shared-файлы сразу.

2. Ветки и PR

Рабочая ветка main постоянно собирается. Короткие ветки feature/P08-financial-confirm, fix/P18-cross-company и docs/P21-readme. Долгоживущая develop и отдельные «frontend-ready/backend-ready» не используются. Ветка живёт обычно до 4-6 часов; незавершённая функция скрывается feature flag, не ломая основной сценарий.

Коммит - одна осмысленная проверяемая правка. Формат feat(applications): confirm saved financial version, fix(access): reject stale company context, docs(demo): add RFI walkthrough. Не коммитить секреты, дампы, пользовательские документы, model caches, node_modules и virtualenv. .env.example содержит имена и безопасные описания, но не действующие значения. Если секрет попал в историю, сначала отозвать/перевыпустить его; удаление строки не устраняет компрометацию.

Перед работой получить свежий main и создать feature-ветку. Rebase допустим только для собственной неразделяемой ветки. В общую ветку и main force-push запрещён. PR маленький и завершённый: проблема/новое поведение, затронутый контракт, проверка и ограничения. Скриншот - дополнение к воспроизводимому сценарию, не замена.

Один независимый reviewer обязателен. Contracts/security/финансовые переходы проверяет назначенный владелец и второй доступный участник при существенном риске. Не ждать завершения всех направлений: frontend использует MSW, backend - ML doubles, ML - synthetic contract fixtures. Такие doubles явно помечаются и не выдаются за работу модели на защите.

3. Контракты и конфликты

Сначала PR контракта: изменение OpenAPI/schema + положительный/отрицательный пример + потребители и связанная задача. После merge потребители обновляют generated types и заглушки. В 1.4 нет live-клиентов, поэтому несовместимый design-change разрешён согласованно, но не тихим изменением wire-shape в одном направлении.

Номер миграции назначает backend-владелец; уже применённая миграция неизменна, исправление - следующей миграцией. Общий контракт не разрешается конфликт-стратегией «взять моё»: владелец объединяет смысл и проверяет обе группы примеров. Сначала исправлять авторский источник, потом повторно генерировать types/PDF/diagram.

Если конфликт в собственной ветке: получить main, обновить свою ветку, разрешить руками, проверить diff и прогнать профильные проверки. Не удалять чужие файлы, чтобы «прошла сборка». Неразделяемую переписанную feature-ветку при необходимости публиковать с force-with-lease; main и ветки других участников так не обновляются.

4. Автоматические проверки

На каждом PR: сборка и типы затронутого кода, проверка формата без исправлений, OpenAPI/JSON Schema/ref/совпадение generated-копий, поиск секретов и проверка изменённых зависимостей. Для backend-переходов - PostgreSQL-тесты CAS/outbox/idempotency; для frontend - профильный сценарий формы; для ML - transforms/parity/shape/digest и evaluation report. Документационный PR не обязан переобучать модели.

На main и перед release: integrated Docker smoke, seed, основной путь, RFI, negative access, restart и проверка 4 реальных ML-функций. Dependency/OCI/model digests входят в SBOM/release manifest. Проверка внешней scanner DB и весов выполняется при подготовке образов, не во время пользовательской операции.

Правила branch protection: PR required, минимум 1 approval, required CI checks, запрет force-push/delete main. Если выбранный тариф/visibility не поддерживает нужную защиту, это отмечается явно и вводится процедурный peer-check; не писать, что техническая защита включена без проверки настроек. В этой поставке настройки аккаунта не менялись.

5. Интеграционный ритм

0-2 ч: контракт/распределение ответственности и синтетические IDs. К 12 ч: один реальный путь web→backend→DB→симулятор→status, с сохранением после restart. На первом срезе допустим подготовленный seed подтверждённых финансов и явно обозначенные ML doubles; запреты версии/прав не обходятся «временным» endpoint.

Каждые 4-6 часов - короткий общий прогон main и фиксация сломанного шага с владельцем. Ошибка интеграции важнее новой карточки/анимации. К 24 ч должны быть зафиксированы API финансов/RFI, CLEAN-путь файлов и выбранные/запускающиеся CPU-модели. К 36 ч - подключение реальных ML-функций; при отставании пересмотреть capacity и публично зафиксировать риск.

Для окна 72 ч заморозка новых функций на 60 ч, следующие 8 ч - регрессия/README/чистый запуск/репетиция, последние 4 ч - версия/проверка доступа/резерв. Для окна 48 ч заморозка на 36 ч. Полный согласованный объём не объявляется готовым, если после заморозки реальные ML заменены заглушками.

6. Веса, лицензии и поставка

Артефакт модели имеет repository/revision/file, SHA-256, licence file, code/preprocessor/feature schema, evaluation digest и approved-by. «latest», имя папки или один номер modelVersion без содержимого недостаточны. Большой bundle хранится вне Git с публичным бесплатным источником и контрольной суммой; для offline-показа комплект готовится заранее. Git LFS разрешён только если жюри действительно может скачать содержимое без оплаты/корпоративного доступа.

Исследовательский UCI-датасет описывается отдельно от synthetic demo seed. THIRD_PARTY_NOTICES содержит версии/лицензии фактически включённых компонентов, моделей, traineddata, datasets и перечень симулируемых IAM/БКИ/ФНС/банковского

конвейера/оформления. Список названий без проверки фактических бинарников не закрывает лицензирование.

6a. Провайдеры и каркасы сервисов

Порты и contracts/technology-adapters.json фиксируются в P01; локальные backend-адаптеры - P25, ML feature/vector/generation - P11/P14, registry/job/quality/metrics bindings - P26. SDK-адаптеры хранятся отдельными модулями с явным статусом. Выбор неготового каркаса обязан остановить запуск с ProviderNotImplemented. PR нового провайдера включает binding, SDK/server locks, общий контрактный набор и provider-specific failure tests. Общие инварианты не ослабляются ради особенностей SDK.

На сдачу сохраняются фактически выбранные providers, digest конфигурации и ссылки/hashes результатов. Статусы LOCAL_SUBSTITUTE, REAL_CONNECTED, TEST_DOUBLE и ARCHITECTURAL_SLOT отражают факты. Настоящий сервис без вызывающего его адаптера остаётся архитектурным слотом. Неподключённая интеграция получает NOT_EXECUTED; демонстрация её локального заменителя не становится PASS для сервиса.

Часть II. Подготовка к оценке

7. Самостоятельный запуск экспертом

README должен содержать: назначение и границы; architecture/context diagram; требования к CPU/RAM/disk/OS/Docker; подготовку бесплатных артефактов и их hashes; один основной способ запуска; переменные окружения; healthchecks/порты; synthetic accounts; пошаговый happy/RFI/ML сценарий; сброс только demo data; ограничения; API/состояния/модель данных/тесты; версию сдачи.

Целевой интерфейс будущего запуска: docker compose -f deploy/compose.yaml up --build -d. Установка models/bundle выполняется один раз по README до запуска. После подготовки модели и данные не скачиваются в runtime. Предусмотреть offline docker load/model bundle, проверяемый SHA-256, и отдельный безопасный reset-script с предупреждением об удалении только demo-volumes. Эти команды являются требованиями к будущему приложению: готового compose в архитектурном комплекте нет.

Для проверки подготовить 2 synthetic компании A/B и представителя каждой, отдельную demo-operator identity, 3 продукта, 10 вариантов DemoFinancial v1, 15-30 KB статей. Не использовать действующие пароли/ИНН/документы людей. Демо-credentials могут быть открыты только в локальном профиле с синтетикой; для внешнего показа - отдельная конфигурация, HTTPS и ограниченный доступ.

За два этапа до сдачи второй участник на чистой машине выполняет README без подсказок автора. Записываются OS/architecture/resources, hash релиза, команды и фактическое время startup/warmup/основного сценария. «У меня запускается» не заменяет этот протокол.

8. Пять минут защиты

Время	Показ	Что доказываем
0:00-0:30	Предпринимателю нужны деньги; открыть synthetic компанию и выбор	Понимание задачи и продукта
0:30-1:15	Диагностика, пригодные продукты, объяснение порядка	Связь eligibility и реальных semantic-рекомендаций
1:15-2:00	Финансовая форма, OCR одного файла, исправление/подтверждение	Снижение ввода при контроле значимых значений
2:00-2:50	Отправка, сохранённый статус, запрос одного уточнения и ответ	Работающий конвейер и небинарная логика
2:50-3:30	Показать изменение ленты и pinned next action; RAG со ссылкой	Персонализация по контексту без потери обязательного шага
3:30-4:10	Результат, альтернативные условия, принятие; demo-завершение	Финансовая логика и честное моделирование операций
4:10-5:00	Отдельно 1 сбой/защита, схема Qdrant/Redis/Feast → порты → выбранные адаптеры	Безопасность, надёжность, путь развития

До показа прогреть модели и корпус, но не подменять RAG заранее заготовленным ответом. При медленном CPU начать вопрос раньше и продолжить показ заявки; если модель не ответила, честно показать SEARCH_ONLY. Основной live-сценарий управляется реальными действиями; подготовленные сценарные seed допустимы и обозначены. Видео - резервное доказательство, не замена доступному прототипу.

9. Матрица критериев жюри

Критерий кейса	Где демонстрируем	Доказательство
Целостность	От выбора до результата/оформления	A01/A02, один applicationId/caseId, persisted history
UX	Предзаполнение, сохранение, ясный статус/RFI	A07/A16/A31, keyboard check и наблюдения тестера
Согласованность модулей	OCR→confirm→snapshot→risk→decisio n	Contracts1.4, traceId и immutable refs, A05/A28
Enterprise-подход	Tenant isolation, CAS/outbox, fail-closed files	A08-A15/A24/A25, результаты реальных проверок
Потенциал развития	Переход от local-mvp к bank-target	Схема полного стека, provider manifest, A38-A43, gates G01-G08, ограничения локальных замен
Готовность к проверке	Чистый Docker-запуск и synthetic credentials	P20/A30/A35, README и hash версии

Не подменять результаты приложения статическими PASS этого архитектурного комплекта. После разработки runtime-протокол заполняется фактическими данными отдельно.

10. Типовые вопросы

Почему модульный монолит? Одна транзакционная граница и меньше затрат на интеграцию для небольшой команды. Отдельные Python/LLM/worker процессы сохраняют возможность масштабировать тяжёлые задачи. Выделение остальных сервисов обосновывается измерениями.

Где настоящий ML? Logistic baseline на открытом исследовательском наборе; semantic ranking на pretrained E5; Tesseract над байтами загруженной формы; retrieval+generation локальной открытой LLM. Показать bundle/revision/evaluation. Правила банка и eligibility - отдельная логика.

Это настоящая вероятность дефолта? Нет: BANKRUPTCY_PROXY, переносимость на российский малый бизнес не подтверждена. Нельзя обещать предотвращённые потери или PD-калибровку.

Что при падении модели? Risk→MANUAL, ranking→RULES, OCR→ручной ввод, LLM→SEARCH_ONLY при доступных источниках. Потеря KB authority закрывает помощник. Fallback-показ не заменяет отдельную демонстрацию рабочих моделей.

Почему нет полного MLOps? Для статических bundle и малого набора запуск платформы не устраняет главный риск - отсутствие работающего пути. Target-платформа сохранена с условиями включения.

Как защищены деньги? Реального перевода нет. В target решение/предложение/подпись/выдача принадлежат банку; повтор бизнес-эффекта контролируется ключами, CAS и сверкой. Не заявлять exactly-once delivery.

11. Фиксация версии и сдача

До дедлайна: merged main, зелёная обязательная CI, clean startup, licenses/README/demo evidence, commit SHA в release manifest, аннотированный тег hackathon-submission-v1.0.0 на этом SHA. Репозиторий не требуется публиковать в облако для соответствия обязательному local-launch; публичный стенд - дополнительный канал.

Проверить, что тег доступен на remote и указывает на нужный commit, а команда запуска воспроизводится именно с него. Изменения после дедлайна не приписывать исходной сдаче. Не перемещать уже опубликованный submission tag; исправление - отдельный тег и явное описание с учётом правил организаторов.

Для приватного репозитория кейс указывает аккаунт жюри alfa-hack-2026. Перед предоставлением доступа сверить имя с актуальными инструкциями организаторов; проверить доступность кода, release assets и всех LFS objects. Создание приглашения и отправка материалов выполняются владельцем/по отдельному поручению, здесь не выполнялись.

Сохранить submission record: commit/tag, время дедлайна с timezone, ссылки на репозиторий/артефакты, licence inventory, hash bundle, протокол чистого запуска и список фактических ограничений. Критичные незакрытые проверки перечислить прямо. Не отправлять в жюри архивы исходных пользовательских документов/исследовательских материалов, если они не нужны для воспроизведения и нет права распространения.

12. Резервный сценарий

Подготовить тот же hash версии на локальном ноутбуке: заранее загруженные images/models, synthetic seed, проверенные offline fonts и PDF. При потере внешнего стенда открыть локальный integrated профиль. При недостатке GPU показать CPU без изменения модели. При сбое live runtime назвать сломанный шаг и показать ранее записанный прогон отдельно, затем оставить жюри инструкции воспроизведения. Не выдавать запись за текущую интерактивную работу.

Технологии на защите: проверяемый рассказ

Отдельное руководство показывает полный mapping. За 20 секунд объяснить: «Qdrant предусмотрен для векторного индекса; на этом CPU-стенде VectorIndexPort использует настоящий NumPy cosine search. Redis имеет отдельные порты сессий и кеша; локально они работают через JDBC и TTL-кеш. Feast-граница сохраняет immutable признаки и историческую доступность; полноценная materialization ещё не реализована». Затем показать один фактический вызов и его evidence. Это архитектурная готовность к подключению, не заявление об интеграции неработающего сервиса.

Если P27 выполнен, показать selectedProvider, изменение метрики/результата настоящего сервиса и протокол отказа. Если времени нет, показать отключённый каркас, конкретный интерфейс и общий набор проверок со статусом NOT_EXECUTED для SDK. Ни UI-скриншот сервиса, ни пустой Docker-контейнер не подтверждают интеграцию.